UNIT – III: AGILE PROCESS MODELS & AGILITY IN REQUIREMENTS ENGINEERING

1. Describe the SCRUM framework and identify its major components.

Scrum is a lightweight Agile framework used to manage complex software development. It divides development into short, fixed-length iterations called **Sprints**, usually lasting 1–4 weeks. Scrum is based on **transparency, inspection and adaptation**.

Major components of Scrum:

1. Product Backlog:
It is a continuously updated and prioritized list of all requirements and features needed in the product.

2. Sprint Backlog:
It contains the selected Product Backlog items that the team plans to complete in the current Sprint.

3. Product Increment:
It is the working and potentially shippable product produced at the end of a Sprint.

Scrum Roles:

- **Product Owner:** Manages and prioritizes the Product Backlog and maximizes product value.
- **Scrum Master:** Guides the team in Scrum practices, removes obstacles and facilitates Scrum events.
- **Development Team:** A cross-functional and self-organizing team that develops the working product.

Scrum Events:

1. Sprint Planning
2. Daily Scrum
3. Sprint Review
4. Sprint Retrospective
5. Sprint

During a Sprint, the team develops and tests the selected work. At the end, the working Increment is demonstrated to stakeholders and feedback is collected.

Conclusion:
Scrum helps teams deliver working software regularly, receive continuous feedback and adapt to changing requirements.

---

2. Illustrate the lifecycle of Extreme Programming (XP).

Extreme Programming (XP) is an Agile method mainly focused on producing high-quality software through strong engineering practices. Its lifecycle consists of six phases.

XP Lifecycle:

1. Exploration:
Customers provide rough **user stories**. The team explores technology and architecture options and gives an initial estimate.

2. Planning – Planning Game:
Customers and developers work together to prioritize user stories and decide the scope and date of the first release.

3. Iterations to Release:
The release is divided into short iterations, normally **1–3 weeks**. Each iteration produces working and tested software. High-priority stories are developed first.

4. Productionizing:
Before the first release, additional testing and performance checks are performed to ensure that the system is ready for production.

5. Maintenance:
After release, the team maintains the system while continuing to develop new features through further iterations.

6. Death:
The project ends when there are no more valuable stories to develop or when the system is retired. Final documentation is completed.

Flow:

Exploration → Planning → Iterations → Productionizing → Maintenance → Death

Conclusion:
XP provides frequent releases, continuous testing and customer involvement throughout development.

---

3. Summarize the lifecycle phases of Adaptive Software Development (ASD).

Adaptive Software Development (ASD) was created by **Jim Highsmith**. It is designed for projects where requirements and technology are uncertain and likely to change.

ASD replaces the traditional **Plan-Build-Revise** approach with a continuous:

Speculate → Collaborate → Learn

1. Speculate

Instead of creating a fixed and detailed plan, the team defines:

- Project mission
- Flexible constraints
- Initial release plan

The plan is treated as a **revisable guess**, because not everything is known at the beginning.

2. Collaborate

Developers, customers and stakeholders work together closely to build the required software components.

Collaboration is important because complex problems cannot always be solved correctly by individuals working separately.

3. Learn

At the end of each cycle, the team reviews:

- Quality of the software
- Project progress
- Team performance

The lessons learned are used in the next **Speculate** phase.

ASD Cycle:

Speculate → Collaborate → Learn → Speculate → Collaborate → Learn

The three phases overlap and repeat continuously throughout the project.

Conclusion:
ASD is suitable for high-uncertainty projects because it allows the team to continuously adapt based on learning and feedback.

4. Differentiate between traditional requirements elicitation and Agile requirements elicitation.

Requirements elicitation means discovering and understanding what customers and stakeholders need from the software.

Traditional Elicitation	Agile Elicitation
Usually done mainly at the beginning of the project.	Done continuously throughout the project.
Attempts to collect detailed requirements upfront.	Collects only enough detail needed for the next iterations.
Produces a comprehensive formal SRS.	Uses user stories and acceptance criteria.
Stakeholder interaction happens at specific stages.	Stakeholders are involved frequently.
Requirements are comparatively stable after approval.	Requirements can change and be reprioritized.
Change is handled through a formal change-request process.	Changes are normally absorbed through backlog refinement.
Focuses on detailed documentation.	Focuses more on conversations and working software.
Validation is often done through formal reviews/sign-off.	Validation happens continuously through Sprint Reviews and demonstrations.

Agile Elicitation Techniques:

- User story workshops
- Backlog refinement
- Sprint Reviews
- On-site customer access
- Story mapping
- Prototyping
- Observation and contextual inquiry

Conclusion:
Traditional elicitation tries to define requirements upfront, while Agile treats requirements elicitation as a **continuous and collaborative activity**.

---

5. Analyze the challenges involved in managing changing requirements in an Agile project.

Changing requirements are common in Agile because customers may understand their needs better after seeing working software. Agile does not try to completely eliminate changing requirements; instead, it provides techniques to manage them safely.

Major challenges:

1. Scope Creep:
New requirements may continuously be added without removing lower-priority work. This can overload the team.

2. Thrashing:
The team may repeatedly start and abandon features when priorities keep changing.

3. Requirements Churn:
Requirements may change repeatedly without understanding the reason for the change, causing the same mistakes to occur again.

4. Technical Debt:
Frequent changes may lead to rushed code modifications and rework, reducing software design quality.

Agile techniques to manage changes:

- **Rolling-wave / Just-in-Time Elaboration:** Detail only the requirements needed for the next one or two iterations.
- **Backlog Reprioritisation:** New or changed requirements can be added and prioritized in the Product Backlog.
- **Timeboxing Change:** Protect the current Sprint from disruptive changes and handle changes between iterations.
- **Spikes:** Conduct short investigations to reduce uncertainty.
- **INVEST Criteria:** Stories should be Independent, Negotiable, Valuable, Estimable, Small and Testable.
- **Continuous Stakeholder Engagement:** Regular customer interaction helps identify changes early.
- **Frequent Small Releases:** Small releases reduce the amount of work affected by late changes.

Conclusion:
Agile manages changing requirements through a living backlog, protected iterations, continuous feedback and frequent releases.

---

6. Compare the roles and practices of SCRUM and Extreme Programming.

Scrum and XP are both Agile approaches, but they focus on different areas. **Scrum mainly focuses on managing and organizing work**, while **XP focuses more on engineering and coding practices**.

Roles comparison:

Scrum	Extreme Programming (XP)
Product Owner	Customer
Scrum Master	Coach
Development Team	Programmers
—	Tester
—	Tracker
—	Manager / Big Boss

Scrum Practices:

- Sprint Planning
- Daily Scrum
- Sprint Review
- Sprint Retrospective
- Product Backlog management
- Story points and Planning Poker
- Velocity tracking
- Time-boxing
- Cross-functional collaboration

XP Practices:

- Planning Game
- Small Releases
- Simple Design
- Test-First / TDD
- Refactoring
- Pair Programming
- Collective Code Ownership
- Continuous Integration
- Sustainable Pace
- On-Site Customer
- Coding Standards
- System Metaphor

Main Difference:

Scrum: Focuses on **project management, roles, events and organizing work into Sprints.**

XP: Focuses on **software engineering quality through testing, pair programming, refactoring and continuous integration.**

Conclusion:
Scrum and XP can also be used together. Scrum can provide the management framework while XP practices provide strong engineering discipline.

7. How does Agile accommodate changing customer requirements?

Agile accepts that customer requirements may change during software development. Instead of freezing requirements at the beginning, Agile treats them as a **living and continuously changing backlog**.

Ways Agile accommodates changes:

1. Living Product Backlog:
Requirements can be added, removed, reordered and re-estimated according to customer needs.

2. Continuous Customer Feedback:
Customers and stakeholders regularly review working software and provide feedback.

3. Short Iterations:
Development is divided into short iterations/Sprints, so changes can be included in upcoming iterations.

4. Backlog Reprioritisation:
The Product Owner decides the priority of new or changed requirements based on business value.

5. Just-in-Time Requirements:
Detailed requirements are developed only when they are close to implementation.

6. Sprint Reviews:
Working software is demonstrated to stakeholders, helping discover new or changed requirements.

7. Frequent Small Releases:
Small increments reduce the amount of work that can become invalid because of a late change.

8. Protected Current Sprint:
Normally, a change that arrives during a committed Sprint waits for the next Sprint instead of disturbing ongoing work.

Example:

If a customer requests a new payment option during a project, the requirement can be added to the Product Backlog. The Product Owner prioritizes it, and the team can implement it in a future Sprint.

Conclusion:
Agile accommodates change through short iterations, continuous feedback, backlog reprioritization and frequent delivery rather than treating change as a major disruption.

---

8. List the major practices followed in Extreme Programming (XP).

Extreme Programming follows **twelve major practices**. These practices are designed to improve software quality, communication and responsiveness to change.

Twelve XP Practices:

1. Planning Game:
Customers and developers jointly decide scope and schedule based on business priorities and technical estimates.

2. Small Releases:
Working software is released early and frequently.

3. Metaphor:
A simple shared story or analogy helps the team understand the system.

4. Simple Design:
The system is designed as simply as possible, adding complexity only when required.

5. Test-First / TDD:
Automated tests are written before the production code.

6. Refactoring:
The internal design of the code is continuously improved without changing its external behavior.

7. Pair Programming:
Two programmers work together at one workstation.

8. Collective Code Ownership:
Any team member can improve any part of the code.

9. Continuous Integration:
New code is integrated and tested with the whole system several times a day.

10. Sustainable Pace:
The team works at a pace that can be maintained continuously, avoiding excessive overtime.

11. On-Site Customer:
A real customer representative is available to answer questions and provide clarification.

12. Coding Standard:
The team follows a common coding standard.

XP Values:

The main XP values are:

- **Communication**
- **Simplicity**
- **Feedback**
- **Courage**

Conclusion:
XP combines these practices to achieve high code quality, quick feedback, continuous improvement and better response to changing requirements.